



UNIVERSIDADE FEDERAL DE CAMPINA GRANDE
CENTRO DE ENGENHARIA ELÉTRICA E INFORMÁTICA

Arthur Vieira, Heitor Alves, Isaque Cavalcante, Rafael Alencar

Análise Comparativa do HAProxy e do Nginx

Campina Grande
2026

Arthur Vieira, Heitor Alves, Isaque Cavalcante, Rafael Alencar

Análise Comparativa do HAProxy e do Nginx

Pesquisa apresentada ao Curso de Ciência da Computação da Universidade Federal de Campina Grande para avaliação na disciplina de Sistemas Distribuídos.

UNIVERSIDADE FEDERAL DE CAMPINA GRANDE
CENTRO DE ENGENHARIA ELÉTRICA E INFORMÁTICA

Campina Grande
2026

Sumário

1	INTRODUÇÃO	3
1.1	Contextualização	3
1.2	Tema	3
1.3	Problemas	3
1.4	Objetivos	4
1.4.1	Geral	4
1.4.2	Específicos	4
1.5	Justificativa	4
2	REFERENCIAL TEÓRICO	5
2.1	Balanceamento de Carga e Algoritmos de Roteamento	5
2.2	HAProxy e Nginx	5
2.3	Grafana k6	6
3	METODOLOGIA	7
3.1	Arquitetura e Estruturação do Ambiente	7
3.2	Cenários de Teste e Injeção de Falhas	8
4	RESULTADOS	9
4.1	Baseline	9
4.2	Spike	10
4.3	Failure	11
4.3.1	Falhas de rede	11
4.3.2	Falhas de disponibilidade	12
4.4	Síntese comparativa	14
5	CONCLUSÃO	16
5.1	Limitações	16
	REFERÊNCIAS	18

1 INTRODUÇÃO

1.1 Contextualização

O crescimento acelerado das aplicações web e a demanda por alta disponibilidade consolidaram o uso de balanceadores de carga como componentes críticos na infraestrutura moderna. Entre as soluções mais utilizadas pelo mercado, o HAProxy (HAProxy Technologies, 2026) e o Nginx (F5, Inc., 2026) se destacam pela eficiência e robustez no direcionamento de tráfego. No entanto, a escolha entre essas ferramentas frequentemente levanta dúvidas sobre qual delas entrega o melhor desempenho sob diferentes condições de estresse e volumes de requisições.

A validação de desempenho e a identificação de gargalos em arquiteturas desse tipo exigem a realização de testes de carga rigorosos e reproduzíveis. Nesse cenário, o Grafana k6 (Grafana Labs, 2026) se estabeleceu como uma ferramenta moderna e eficiente para simular fluxos reais de usuários, permitindo mensurar métricas fundamentais como latência, vazão (throughput) e taxa de erros. Assim, a comparação direta entre o HAProxy e o Nginx por meio do k6 justifica-se pela necessidade de obter dados empíricos e mensuráveis que orientem decisões de arquitetura de software mais assertivas e otimizadas para cenários de alta demanda.

1.2 Tema

Análise comparativa do comportamento e desempenho do HAProxy e Nginx como balanceadores de carga submetidos a testes de estresse, variando o volume de requisições e concorrência no direcionamento de tráfego, utilizando a ferramenta Grafana k6

1.3 Problemas

Apesar da ampla utilização do HAProxy e do Nginx como balanceadores de carga em infraestruturas de alta disponibilidade, grande parte das decisões de escolha entre essas ferramentas baseia-se em recomendações genéricas ou conhecimentos empíricos de mercado. Atualmente, há uma lacuna na análise quantitativa direta do comportamento de ambos em cenários idênticos de estresse, considerando a variação de métricas críticas - como latência, vazão (throughput) e uso de recursos - sob diferentes padrões de concorrência e volume de tráfego simulados via Grafana k6. Dessa forma, formula-se a seguinte questão de pesquisa: como o HAProxy e o Nginx diferem em termos de desempenho e eficiência na distribuição de tráfego quando submetidos a testes de carga?

1.4 Objetivos

1.4.1 Geral

Realizar uma análise comparativa do comportamento e desempenho do HAProxy e do Nginx no direcionamento de tráfego por meio da simulação de diferentes cenários de estresse e concorrência utilizando o Grafana k6.

1.4.2 Específicos

- a) Medir quanto o tempo o HAProxy/NGINX leva para detectar e remover do cluster uma réplica que sofreu um crash abrupto (via Passive vs Active Health Checks).
- b) Identificar o impacto no tempo de resposta (latency tail / percentis 95 e 99) e na vazão quando o tráfego sofre um pico repentino (efeito Flash Crowd).
- c) Avaliar como a escolha do algoritmo de balanceamento (ex: Round-Robin vs Least Connections) afeta a recuperação do sistema e a distribuição da sobrecarga residual após a queda de um nó?

1.5 Justificativa

A relevância deste estudo se insere no contexto de consolidação de arquiteturas de alta disponibilidade baseadas em código aberto, complementando a literatura científica existente que aborda o desempenho de balanceadores de carga. Embora investigações prévias tenham examinado algoritmos fundamentais e métricas gerais de redes locais, a presente pesquisa diferencia-se ao integrar de forma combinada o ecossistema de testes de carga do Grafana k6 com a injeção controlada de falhas promovida pela ferramenta Pumba. O estudo avança além de avaliações estáticas ao mapear o comportamento transacional do HAProxy e do Nginx sob picos de requisições concorrentes, fornecendo dados empíricos sobre tolerância a falhas, saturação de recursos e limiares de recuperação.

2 REFERENCIAL TEÓRICO

2.1 Balanceamento de Carga e Algoritmos de Roteamento

O balanceamento de carga é um componente fundamental em arquiteturas distribuídas e de microsserviços, atuando como o ponto central de distribuição do tráfego de rede para otimizar o tempo de resposta, maximizar a vazão e evitar a sobrecarga em servidores individuais. O impacto do balanceador na infraestrutura é diretamente influenciado pela estratégia de roteamento adotada.

Conforme analisado por Pramono, Cokro e Waskito (2018), os algoritmos mais difundidos nas soluções de código aberto são:

- a) **Round Robin (RR)**: Distribui as requisições recebidas de forma cíclica e sequencial entre os servidores da pool, assumindo tacitamente que todas as instâncias possuem a mesma capacidade computacional;
- b) **Least Connections (LC)**: Direciona o tráfego para o servidor que possui o menor número de conexões ativas no momento da requisição, adaptando-se melhor a cenários com requisições de tempo de processamento heterogêneo;
- c) **Weighted Round Robin / Least Connections (WRR/WLC)**: Introduz pesos às instâncias para considerar capacidades de hardware distintas na distribuição.

2.2 HAProxy e Nginx

O HAProxy (HAProxy Technologies, 2026) e o Nginx (F5, Inc., 2026) destacam-se como duas das soluções de código aberto mais robustas e amplamente adotadas para o gerenciamento e balanceamento de tráfego HTTP/TCP. Apesar de cumprirem papéis semelhantes em arquiteturas modernas, ambas as ferramentas possuem características operacionais e filosofias de design distintas:

- a) **HAProxy (High Availability Proxy)**: Projetado especificamente como um proxy reverso e balanceador de carga focado em alta disponibilidade. Destaca-se pelo gerenciamento eficiente de memória, suporte avançado a health checks ativos e pela oferta de estatísticas detalhadas de tráfego em tempo real (HAProxy Stats);
- b) **Nginx**: Desenvolvido originalmente como um servidor web assíncrono e orientado a eventos (event-driven), o Nginx evoluiu para atuar como um poderoso proxy reverso e balanceador de carga. Sua arquitetura baseada em worker processes permite lidar com elevado volume de conexões simultâneas com baixo consumo de recursos.

A escolha entre HAProxy e Nginx em ambientes locais ou distribuídos envolve trade-offs entre consumo de recursos, facilidade de implantação, complexidade de configuração e resiliência. Enquanto o Nginx se destaca pela versatilidade ao servir arquivos estáticos e gerenciar conexões no mesmo processo, o HAProxy oferece mecanismos mais rigorosos para detecção de falhas e isolamento de nós degradados.

O trabalho desenvolvido por Zulman *et al.* (2026) emprega o NGINX como balanceador em uma arquitetura baseada em contêineres Docker para avaliar diferentes estratégias de distribuição de carga. Os resultados demonstram que o balanceador foi capaz de redistribuir automaticamente as requisições durante falhas controladas dos servidores, mantendo a disponibilidade do serviço e reduzindo a degradação do desempenho.

Complementando esses resultados, Ajaz *et al.* (2026) propõem um framework de balanceamento utilizando NGINX em uma rede distribuída local composta por múltiplos servidores. Os experimentos demonstram que a utilização de dois servidores balanceados aumenta significativamente a capacidade de processamento da aplicação quando comparada à utilização de um único servidor, evidenciando os benefícios da distribuição da carga para ambientes sujeitos a grande volume de acessos simultâneos.

2.3 Grafana k6

A avaliação do comportamento de balanceadores de carga exige a geração de tráfego sintético escalável capaz de simular cenários do mundo real (como picos de tráfego, acessos concorrentes e falhas na rede).

O Grafana k6 (Grafana Labs, 2026) é uma ferramenta moderna de testes de carga orientada a desenvolvedores, escrita em Go com suporte à automação de testes em JavaScript. Diferente de ferramentas tradicionais baseadas em interface gráfica, o k6 destaca-se pela alta eficiência na execução de Threads/Virtual Users (VUs) com baixo consumo de CPU e memória, tornando-o ideal para testar limites de vazão e resiliência em balanceadores de carga.

Em uma metodologia de avaliação de balanceamento de carga baseada no k6, destacam-se as seguintes métricas fundamentais:

- a) **Vazão (Throughput / RPS):** Quantidade de requisições por segundo que o balanceador consegue processar e repassar aos backends com sucesso.
- b) **Tempo de Resposta e Latência (Percentis P95 e P99):** O tempo total transcorrido desde o envio da requisição até o recebimento da resposta, com foco na latência de cauda para identificar gargalos sob alta carga.
- c) **Taxa de Erro e Tolerância a Falhas:** Percentual de requisições com falhas (códigos de status HTTP 5xx ou timeouts) durante a injeção controlada de estresse ou indisponibilidade de nós da aplicação.

3 METODOLOGIA

Foram realizados seis experimentos no total, sendo três cenários de teste para cada um dos dois balanceadores de carga analisados, HAProxy (HAProxy Technologies, 2026) e Nginx (F5, Inc., 2026). O objetivo foi comparar o comportamento, a resiliência e a eficiência de ambas as tecnologias sob diferentes perfis de tráfego e condições de falha na infraestrutura. O gerenciamento dos ambientes simulados e a orquestração dos serviços foram realizados por meio do Docker Compose (Docker Inc., 2026), garantindo o isolamento dos componentes e a reprodutibilidade dos testes executados.

A medição quantitativa do desempenho e o monitoramento das requisições foram realizados utilizando a ferramenta de testes de carga Grafana k6 (Grafana Labs, 2026). Para avaliar a qualidade do serviço e a capacidade de resposta dos balanceadores de carga, foram consideradas as seguintes métricas fundamentais:

- a) **Latência:** mede o tempo total transcorrido desde o envio da requisição HTTP até o recebimento completo da resposta. A análise foi realizada considerando os percentis $p(95)$ e $p(99)$;
- b) **Taxa de erro:** avalia a proporção de requisições que resultaram em falha, considerando códigos de status HTTP das classes 4xx e 5xx, além de ocorrências de timeout, em relação ao total de chamadas realizadas;
- c) **Vazão (Throughput):** mensura a taxa de requisições processadas com sucesso por segundo (Requests Per Second – RPS) sustentada pelo balanceador.

3.1 Arquitetura e Estruturação do Ambiente

Para garantir a equidade na comparação, as arquiteturas do HAProxy e do Nginx foram configuradas utilizando condições equivalentes de hardware virtualizado, limites de recursos computacionais e quantidade de nós de backend.

O fluxo de dados da infraestrutura experimental foi organizado em três camadas principais:

- a) **Gerador de carga (Grafana k6):** atua como cliente da arquitetura, realizando o disparo das requisições HTTP simuladas e registrando as métricas coletadas diretamente em disco no formato JSON;
- b) **Camada de balanceamento:** composta pelo container contendo a instância do balanceador de carga sob teste (HAProxy ou Nginx), responsável pela distribuição das requisições recebidas;
- c) **Camada de aplicação (Backends):** formada pelo conjunto de réplicas de microsserviços responsáveis pelo processamento das requisições encaminhadas pelo balanceador.

3.2 Cenários de Teste e Injeção de Falhas

Foram definidos três cenários operacionais para submeter os balanceadores de carga a diferentes regimes de estresse. A injeção controlada de falhas na camada de backend foi realizada utilizando a ferramenta Pumba (Ledenev, 2026), responsável por manipular containers Docker e interfaces de rede por meio de técnicas de *Chaos Engineering*. Os cenários de teste definidos foram:

- a) **Baseline (Linha de Base):** submete o balanceador de carga a um tráfego constante em nível nominal. Esse cenário é utilizado como grupo de controle para estabelecer os limites normais de latência e vazão do sistema sem ocorrência de anomalias;
- b) **Spike (Pico de Tráfego):** caracteriza-se pelo aumento abrupto e massivo da quantidade de usuários virtuais (*Virtual Users – VUs*) simultâneos em pequenos intervalos de tempo. Esse cenário permite avaliar o comportamento relacionado à bufferização, filas de conexões e tempo de recuperação dos proxies;
- c) **Failure (Injeção de Falhas e Degradação de Rede):** executa o perfil de tráfego enquanto distúrbios físicos e operacionais são aplicados ativamente nos containers pertencentes à camada de backend.

Para a execução do cenário de *Failure*, foram selecionadas cinco técnicas de indução de falhas utilizando a ferramenta Pumba. As modalidades aplicadas são apresentadas na Tabela 1.

Tabela 1 – Critérios e modos de falha injetados via Pumba

Técnica	Comando	Alvo	Efeito e objetivo
Abrupt Termination	kill	Processo do container	Interrupção imediata do processo. Avalia a detecção de nós indisponíveis e o redirecionamento para nós saudáveis.
Process Stalling	pause	Container de backend	Congelamento por 10 s. Verifica o gerenciamento de timeouts e filas de conexão.
Graceful Stop	stop	Container de backend	Encerramento controlado por 15 s. Avalia o tratamento de conexões ativas antes da remoção do nó.
Network Latency	netem delay	Interface de rede	Adição de 1000 ms de latência por 30 s. Mede impactos de backends lentos.
Packet Loss	netem loss	Interface de rede	Perda de 20% dos pacotes por 30 s. Avalia degradação da vazão e retransmissões TCP.

4 RESULTADOS

Esta seção apresenta os resultados obtidos na avaliação comparativa entre os balanceadores de carga HAProxy e Nginx, organizados em três cenários de teste — Baseline, Spike e Failure — combinando métricas agregadas de Latência, Taxa de Erro e Throughput com a análise das séries temporais de requisições por segundo.

4.1 Baseline

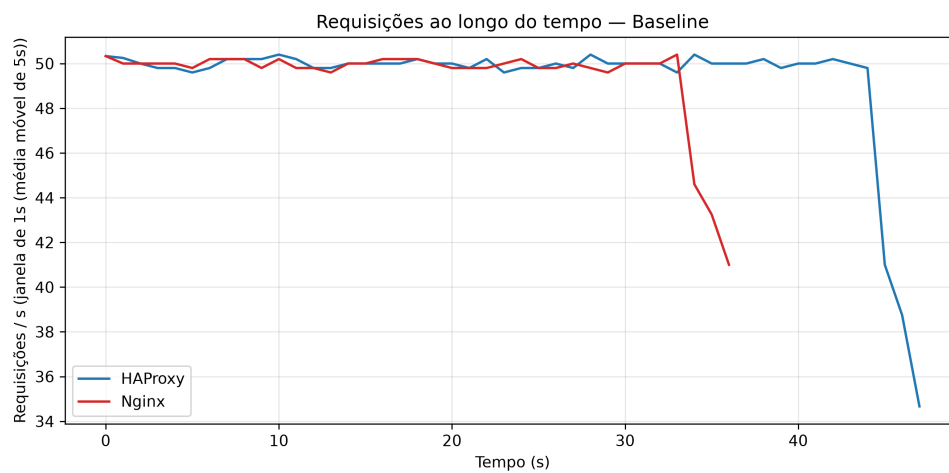


Figura 1 – Requisições ao longo do tempo sem interferência.

Sob carga estável e sem injeção de falhas, os dois balanceadores apresentaram desempenho estatisticamente equivalente. O HAProxy processou 2.355 requisições com latência média de 29,29 ms (P95 = 49,40 ms; P99 = 51,39 ms) e throughput de 50,02 req/s, enquanto o Nginx processou 1.822 requisições com latência média de 29,41 ms (P95 = 50,33 ms; P99 = 51,78 ms) e o mesmo throughput de 50,02 req/s. A taxa de erro foi de 0% para ambos. A série temporal de requisições (Figura 1) confirma essa equivalência: as duas curvas permanecem sobrepostas e estáveis ao longo de todo o teste, sem oscilações relevantes. A queda observada no trecho final de cada curva é um artefato do encerramento gradual das VUs pelo executor de carga (graceful stop) e da perda de suporte da média móvel nas bordas da série — não reflete degradação real de desempenho, e ambos os balanceadores exibem o mesmo padrão de encerramento, apenas em instantes diferentes por terem durações totais de teste distintas.

Esse resultado estabelece o comportamento de referência dos dois balanceadores: sob condições ideais, HAProxy e Nginx são intercambiáveis em termos de latência e throughput, o que reforça que quaisquer divergências observadas nos cenários seguintes são atribuíveis à

resposta a carga elevada ou a falhas, e não a uma diferença intrínseca de desempenho em regime normal.

4.2 Spike

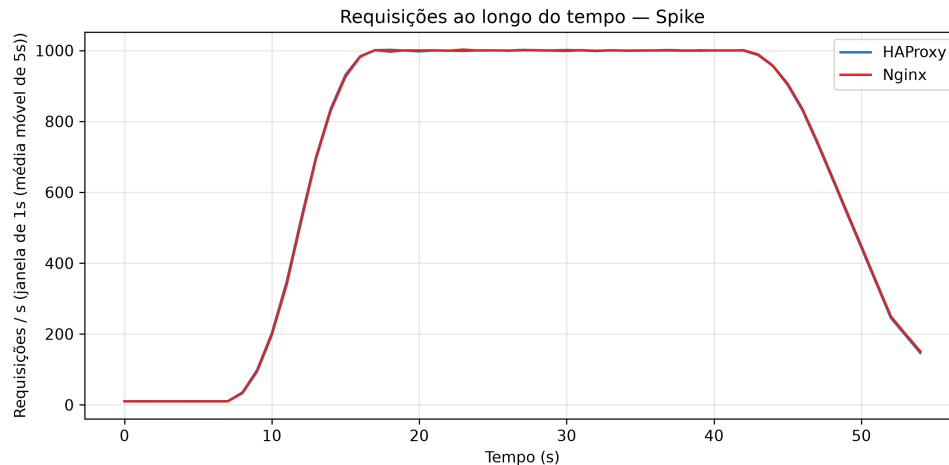


Figura 2 – Pico de requisições ao longo do tempo.

Sob o teste de spike (rampa de carga de 100 para 400 VUs), a curva bruta de requisições recebidas por segundo é praticamente idêntica entre os dois balanceadores (Figura 2), já que o volume de tráfego é definido pelo gerador de carga e não pelo balanceador. A divergência relevante aparece nas métricas de latência e taxa de erro, não no throughput bruto.

Agregando os dois testes de spike (5–50 e 100–400 VUs) por balanceador, o HAProxy processou 58.901 requisições com taxa de erro de 2,53% (1.491 erros), latência média de 1.349,58 ms, P95 de 4.444,02 ms e P99 de 4.742,92 ms. O Nginx processou 74.985 requisições com taxa de erro de 22,67% (16.996 erros), latência média de 282,59 ms, P95 de 634,65 ms e P99 de 679,54 ms. No teste de maior intensidade especificamente (100 para 400 VUs), a taxa de erro do Nginx atingiu 45,55%, contra 7,02% do HAProxy no mesmo teste — uma diferença de mais de 6 vezes.

Esse padrão indica uma diferença qualitativa na forma como cada balanceador lida com saturação: o HAProxy tende a manter as conexões e absorver a sobrecarga sob a forma de latência elevada (fila de espera), resultando em poucos erros, porém tempos de resposta muito altos (P99 de quase 4,7 s). O Nginx, por sua vez, apresenta latências mais contidas nos percentis altos, mas às custas de rejeitar/abortar uma proporção muito maior de requisições sob pico — um comportamento consistente com esgotamento de fila de conexões (backlog) ou de worker connections configurados abaixo do necessário para absorver o pico de 400 VUs simultâneos.

4.3 Failure

O cenário de falha foi decomposto em cinco tipos de interrupção — delay, loss, kill, stop e pause — cada um representando uma classe distinta de degradação de backend (rede vs. disponibilidade do processo).

4.3.1 Falhas de rede

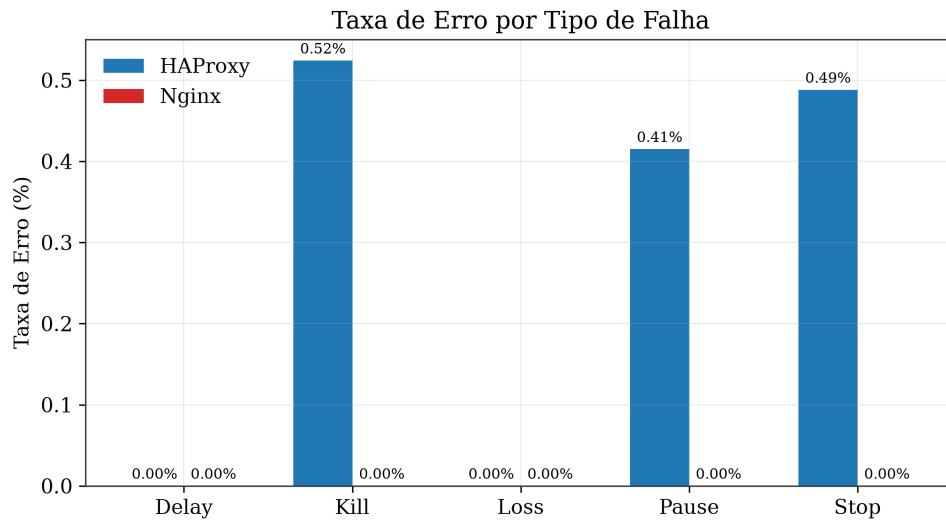


Figura 3 – Taxa de erro por tipo de falha.

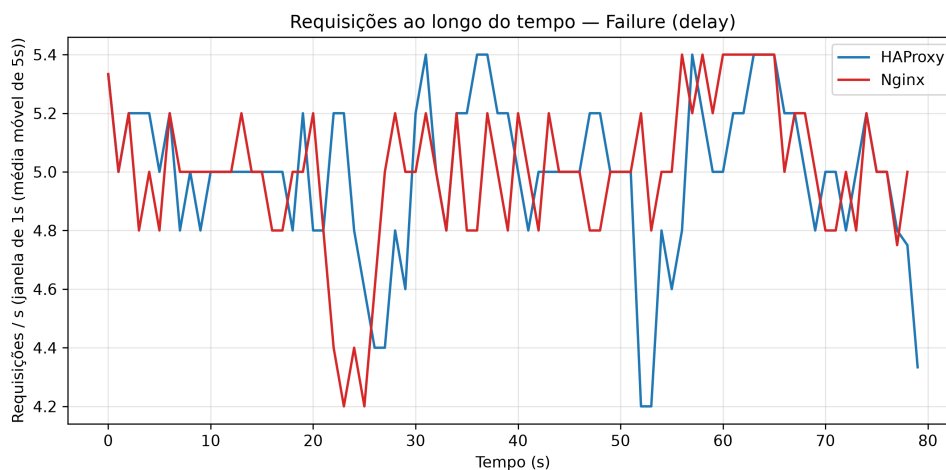


Figura 4 – Requisições ao longo do tempo na falha delay.

Nos dois tipos de falha que afetam apenas a rede — atraso artificial de pacotes (delay) e perda de pacotes (loss) — nenhum dos balanceadores registrou taxa de erro acima de 0% (Figura 3). As séries temporais correspondentes (Figuras 4 e 5) mostram apenas oscilação de alta frequência em torno da taxa nominal de aproximadamente 5 req/s, sem quedas sustentadas

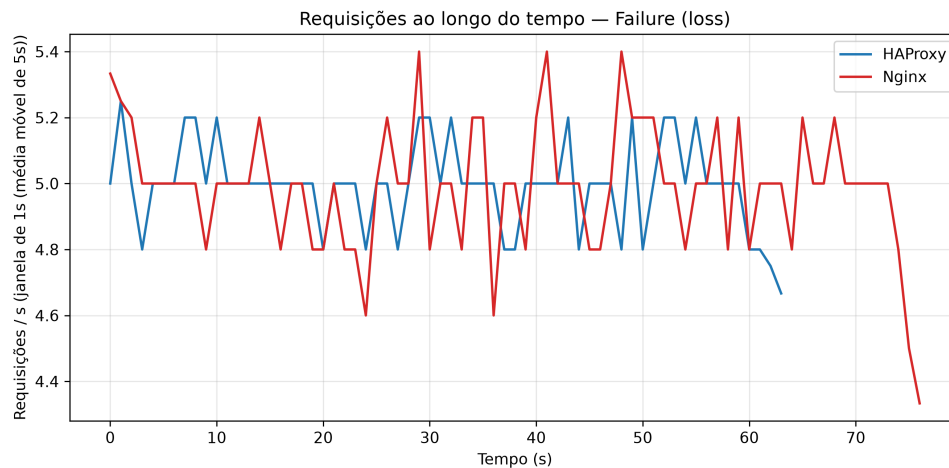


Figura 5 – Requisições ao longo do tempo na falha loss.

de throughput. Esse resultado é esperado: perda de pacote é mitigada pela retransmissão da camada de transporte (TCP), e atraso de rede aumenta a latência (HAProxy: 130,71 ms em delay e 33,84 ms em loss; Nginx: 344,21 ms em delay e 67,07 ms em loss) sem jamais tornar o backend indisponível do ponto de vista do balanceador.

4.3.2 Falhas de disponibilidade

Já nos cenários em que o processo do backend é efetivamente interrompido, o padrão se inverte: o HAProxy foi o único balanceador a registrar erros — 0,52% em kill, 0,49% em stop e 0,41% em pause — enquanto o Nginx manteve 0% de erro nos três casos (Figura 3). As séries temporais explicam o mecanismo por trás dessa diferença:

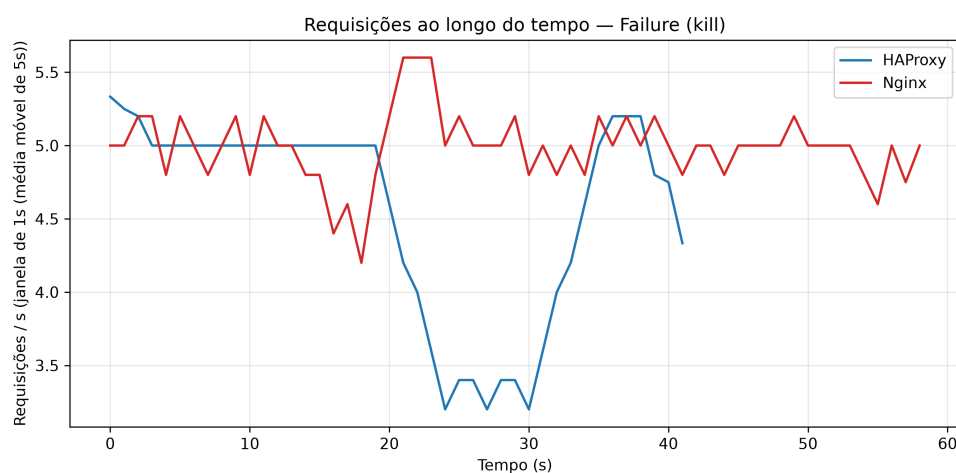


Figura 6 – Requisições ao longo do tempo na falha kill.

- a) Em kill (Figura 6), o HAProxy apresenta um vale profundo e sustentado, caindo de 5 para 3,2 req/s entre aproximadamente $t=20$ s e $t=30$ s — uma janela de recuperação de

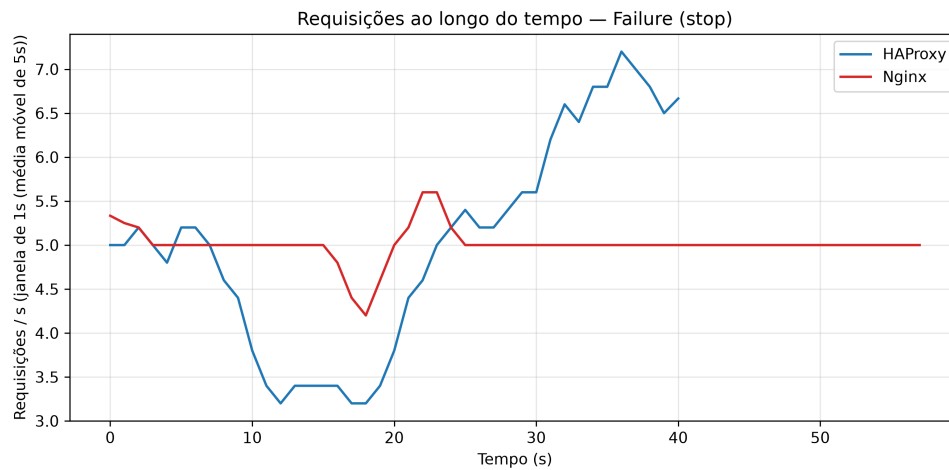


Figura 7 – Requisições ao longo do tempo na falha stop.

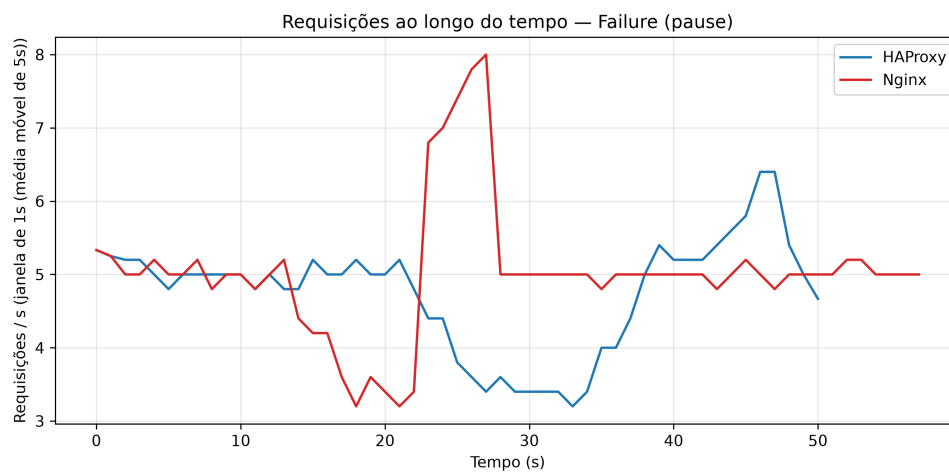


Figura 8 – Requisições ao longo do tempo na falha pause.

cerca de 10 segundos. O Nginx, no mesmo intervalo, mostra apenas uma oscilação mais suave, sem colapso comparável.

- b) Em stop (Figura 7), o mesmo padrão se repete: vale mais profundo e mais longo no HAProxy (queda a 3,2 req/s entre $t=10$ s e $t=20$ s) frente a uma queda mais rasa no Nginx (até 4,2 req/s). Após a recuperação, a curva do HAProxy ultrapassa o patamar nominal, chegando a 7,2 req/s — um overshoot atribuído a VUs que ficaram bloqueadas aguardando timeout durante a falha e dispararam suas próximas iterações em lote assim que o backend volta a responder (artefato do método de geração de carga, não do balanceador).
- c) Em pause (Figura 8), os dois balanceadores respondem de formas distintas ao mesmo estímulo: o Nginx exibe um pico anômalo, subindo de 5 para 8 req/s entre $t=20$ s e $t=27$ s — consistente com o represamento de conexões durante a suspensão do processo (SIGSTOP) seguido da liberação simultânea de respostas quando o processo é retomado. O HAProxy, em contraste, mostra um vale longo e raso (3,2–3,4 req/s entre $t=15$ s e

t=33 s), sugerindo que as requisições expiram por timeout ao longo do tempo em vez de serem enfileiradas — o que é consistente com a taxa de erro de 0,415% observada exclusivamente nesse balanceador neste cenário.

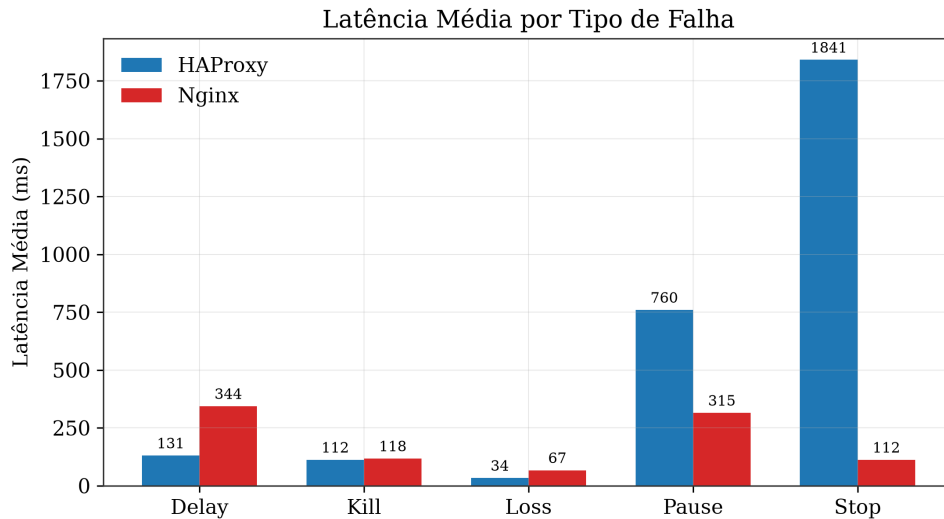


Figura 9 – Latência por tipo de falha.

Em latência, o HAProxy também apresentou os valores médios mais altos nos cenários de disponibilidade — 759,86 ms em pause e 1.840,75 ms em stop — contra 315,08 ms e 112,34 ms do Nginx, respectivamente (Figura 9). Combinado com o padrão das séries temporais, isso indica que o HAProxy, nesta configuração, tende a manter requisições pendentes por mais tempo antes de desviar tráfego para um backend saudável ou expirar a conexão, resultando em uma janela de degradação mais longa, porém com impacto discreto sobre a taxa de erro total.

4.4 Síntese comparativa

A Figura 10 consolida os quatro indicadores centrais (throughput, taxa de erro, latência média e latência P99) por cenário. O padrão geral é consistente: os dois balanceadores são equivalentes em condições normais (Baseline); sob alta concorrência (Spike), o HAProxy prioriza manter a conexão à custa de latência, enquanto o Nginx rejeita mais requisições, mas com latências de cauda mais contidas; sob falha de disponibilidade do backend (Kill/Stop/Pause), o HAProxy leva consistentemente mais tempo para detectar e reagir à indisponibilidade, gerando janelas de degradação mais longas e a totalidade dos erros observados nesses três cenários.

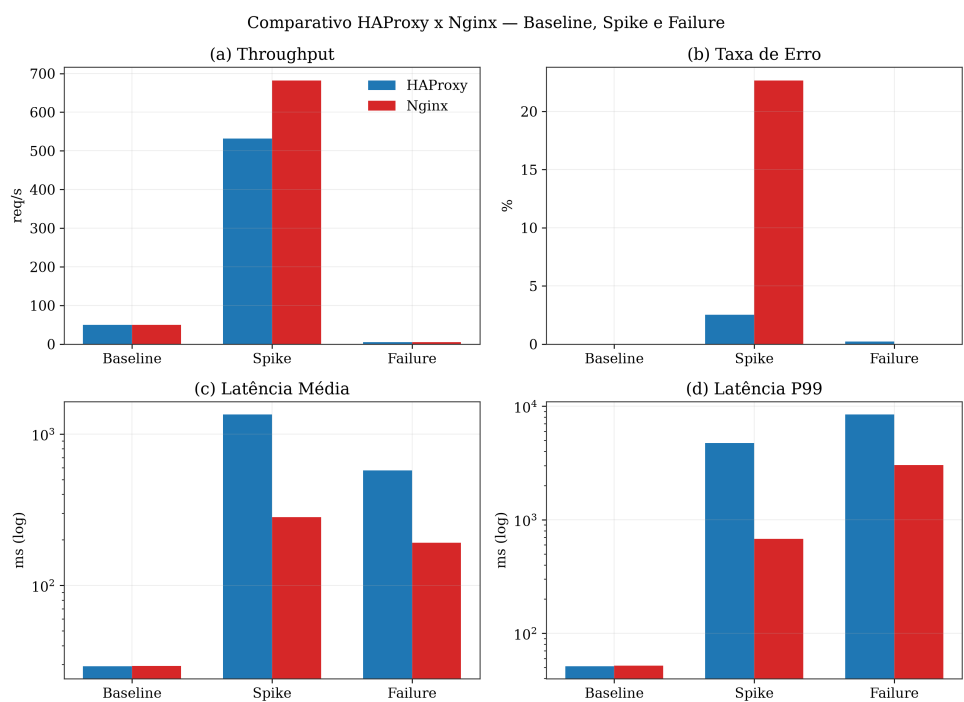


Figura 10 – Resumo Comparativo das métricas de throughput, taxa de erro, latência média e latência P99.

5 CONCLUSÃO

Este estudo comparou o comportamento dos balanceadores de carga HAProxy e Nginx sob três condições — operação normal (Baseline), pico de demanda (Spike) e falhas de backend (Failure, subdivididas em delay, loss, kill, stop e pause) — avaliando Latência, Taxa de Erro e Throughput, complementados pela análise de séries temporais de requisições por segundo.

Os resultados indicam que, em condições normais de operação, os dois balanceadores apresentam desempenho equivalente, validando-os como alternativas intercambiáveis nesse regime. As diferenças relevantes emergem apenas sob estresse: sob pico de carga, o HAProxy demonstrou uma estratégia de absorção de sobrecarga baseada em enfileiramento — poucos erros (2,53%), porém latências de cauda elevadas (P99 próximo de 4,74 s) —, enquanto o Nginx apresentou o comportamento oposto, com latências de cauda mais contidas, mas uma taxa de erro substancialmente maior (22,67%, chegando a 45,55% no teste de maior intensidade). A escolha entre essas estratégias tem implicações diretas de projeto: aplicações que toleram latência elevada, mas não toleram falhas visíveis ao usuário, tendem a se beneficiar do comportamento observado no HAProxy; aplicações que preferem falhar rápido (e tratar o erro no cliente, por exemplo via retry) podem se beneficiar do perfil do Nginx.

Sob falhas que afetam apenas a rede (delay e loss), nenhum dos balanceadores apresentou erros, confirmando que esse tipo de falha se manifesta exclusivamente como aumento de latência. Já sob falhas que derrubam efetivamente o processo do backend (kill, stop e pause), o HAProxy foi consistentemente mais lento para detectar a indisponibilidade e redirecionar o tráfego, resultando em janelas de degradação mais longas nas séries temporais e concentrando a totalidade dos erros observados nesses cenários — ainda que em magnitude reduzida (0,41% a 0,52%). Esse achado aponta para uma provável diferença nos parâmetros de health check e/ou política de retry entre as duas configurações testadas, e não necessariamente para uma limitação arquitetural do HAProxy em si.

5.1 Limitações

Os resultados aqui reportados derivam de uma única execução por cenário, sem repetições que permitam estimar variância entre execuções — recomenda-se, para trabalhos futuros, repetir cada cenário múltiplas vezes e reportar intervalos de confiança sobre as métricas agregadas. Adicionalmente, os efeitos de borda identificados nas séries temporais (queda no encerramento do Baseline, overshoot pós-falha em stop) são artefatos do método de geração de carga (VUs bloqueadas por timeout retomando iterações em lote) e não do balanceador avaliado; embora tenham sido identificados e discutidos, sua magnitude não foi isolada quantitativamente.

Em síntese, a escolha entre HAProxy e Nginx neste estudo não aponta para um vencedor

absoluto, mas para um trade-off explícito entre tolerância a latência sob pico (favorecendo HAProxy) e velocidade de detecção de falha de backend (favorecendo Nginx nesta configuração) — uma decisão que deve ser guiada pelos requisitos específicos de disponibilidade e experiência do usuário de cada aplicação.

REFERÊNCIAS

AJAZ, Ubaid *et al.* Optimizing Load Balancing Framework for a Distributed Local Network. **International Journal on Perceptive and Cognitive Computing**, v. 12, n. 1, p. 131–136, 2026. DOI: 10.31436/ijpcc.v12i1.679. Disponível em: <https://journals.iium.edu.my/kict/index.php/IJPCC/article/view/679>.

DOCKER INC. **Docker Compose**. [S. l.: s. n.], 2026. Disponível em: <https://docs.docker.com/compose/>.

F5, INC. **NGINX**. [S. l.: s. n.], 2026. Disponível em: <https://nginx.org/>.

GRAFANA LABS. **Grafana k6**. [S. l.: s. n.], 2026. Disponível em: <https://k6.io/>.

HAPROXY TECHNOLOGIES. **HAProxy: The Reliable, High Performance TCP/HTTP Load Balancer**. [S. l.: s. n.], 2026. <https://www.haproxy.org/>.

LEDENEV, Alexei. **Pumba: Chaos Testing and Network Emulation for Docker Containers**. [S. l.: s. n.], 2026. Disponível em: <https://github.com/alexei-led/pumba>.

PRAMONO, Luthfan; COKRO, Robby; WASKITO, Yanuar. Round-robin Algorithm in HAProxy and Nginx Load Balancing Performance Evaluation: A Review. *In: PROCEEDINGS of the 1st International Seminar on Research of Information Technology and Intelligent Systems (ISRITI)*. [S. l.: s. n.], nov. 2018. p. 367–372. DOI: 10.1109/ISRITI.2018.8864455.

ZULMAN, Muhammad Reza *et al.* Performance and Fault Tolerance Analysis of Load Balancing Strategies in Microservices Architecture with Controlled Failure Injection. **SISFO: Jurnal Ilmiah Sistem Informasi**, v. 10, n. 1, p. 9–18, 2026. DOI: 10.29103/sisfo.v10i2.26893. Disponível em: <https://ojs.unimal.ac.id/sisfo/article/view/26893>.